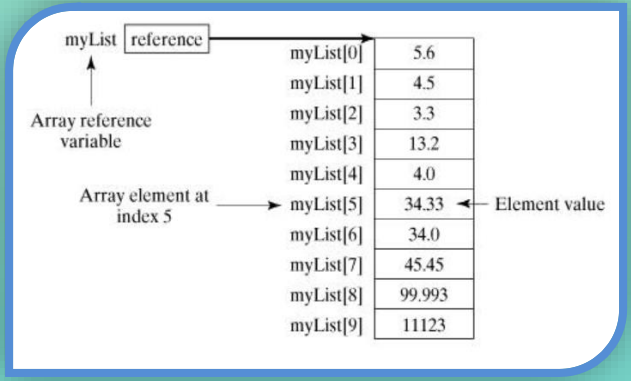
Arrays
Java Lecture-1

Agenda

- Arrays in java
  - Ways to Initialize Arrays
  - Array Index Out Of Bounds Exception
  - Arrays vs Collection Framework
- Quiz
- Homework

Arrays & their Initialization

- An **array** is a container that holds **multiple data (values)** of one **single type**
- A Java **array variable** can also be declared like other variables with **[]** after the data type
- **Arrays** are **fixed** in size i.e. their size is declared at the time of their **initialization**
- **Arrays** are **object** in java and direct **superclass** of an array type is **java.lang.Object**
- Arrays can be of **primitive data type** as well as of **Class type**
- Arrays can be initialized in following ways:
  - **Array in a single statement:**
    int[] arr1 = {3, 5, 7, 9, 11};
    - **[] square brackets** can be put **after data-type** or even after **variable name**
    - Length of the above array can be calculated by counting number of elements present




```
int newArr[] = {12, 15, 18, 22}; //primitive array with elements in one step
System.out.println("value at index[1] is: "+newArr[1]);
System.out.println("value at index[2] is: "+newArr[2]);
//non-primitive array
String[] schoolbag2 = { "Books", "Pens", "Pencils", "Notebooks" };
System.out.println("The third element is: " + schoolbag2[2]);
System.out.println("size of array: "+schoolbag2.length);
```

More Ways to Initialize Arrays

- **Declaring an array with their size during initialization:**

  int arr2[] = new int[3];

  - Index of **array** always start from **0**. So, above array will have **index 0, 1, 2**
  - **Default value** of element at every index is **0** in case of **primitive** array and **null** in case of **non-primitive** array
  - **Length** of an array is always equal to **last index+1**
  - **Elements** in above array can be accessed as **arr2[0]=5;arr2[1]=10;arr2[2]=20;**
  - **length member variable** – It is used to get size or **number of elements** of an array

- **Declaring an array without size and with "new" keyword:**

  int arr3[] = new int[]{3, 5, 7, 9};

  - Here, we are **initializing** an array with **"new"** keyword and then declaring elements in single line
  - Here, we are declaring length of an array indirectly when elements are declared directly in a single statement

```
//int array(primitive)
int[] intArray = new int[3];
intArray[0] = 10;
intArray[1] = 20;
intArray[2] = 30;
for (int i=0; i<3; i++)
{
    System.out.println(intArray[i]);
}
//String array(non-primitive)
String[] stringArray = new String[3];
stringArray[0] = "a";
stringArray[1] = "b";
stringArray[2] = "c";
System.out.println("stringArray output");

for (int i=0; i<stringArray.length; i++)
{
    System.out.println(stringArray[i]);
}
//Declaring an array without size and "new" keyword
int[] intArray2 = new int[] {4,5,6,7,8};
System.out.println("intArray output (version 2)");

for (int i=0; i<intArray2.length; i++)
{
    System.out.println(intArray[i]);
}
```

Array Index Out Of Bounds Exception

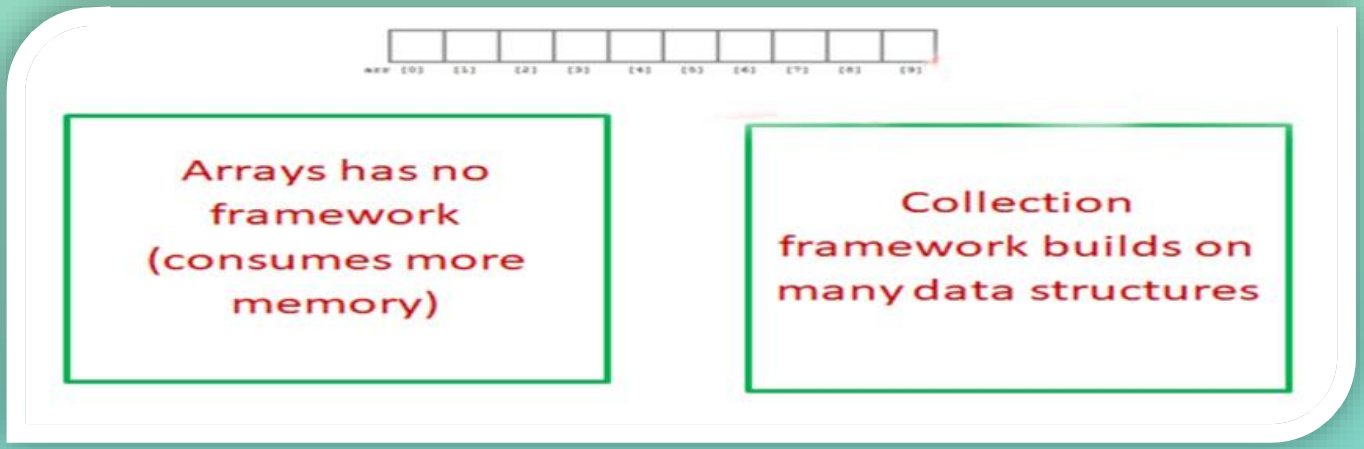
- If an **array** is accessed with negative index or an **index** greater than or equal to size of an **array**, then the JAVA throws a **ArrayIndexOutOfBounds Exception**
- **ArrayIndexOutOfBoundsException** is a Runtime **Exception** thrown **only at runtime** and the Java Compiler **does not** check for this error during the compilation of a program
- One should always note that **index of an array** always starts from **0**

```
int[] intArray = new int[3];
intArray[0] = 10;
intArray[1] = 20;
intArray[2] = 30;
for (int i=0; i<=3; i++)
{
    System.out.println(intArray[i]);
}
```

This code will throw below exception

```
10
20
30
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 3
```

Arrays vs Collection Framework



- **Following are main points of difference between Arrays and Collection Framework**

Arrays	Collection Framework
Arrays are static(fixed) in size	It is dynamic in size (expands & collapse automatically)
Arrays can store only homogenous elements	It can hold heterogenous elements
Arrays can hold both primitive and objects	Collection can hold only objects

Quiz

- How will you declare an **array** variable?
- How will you calculate **size of an array** (with method or member variable)?
- Which **exception occurs** when you access element **outside index** of an array?

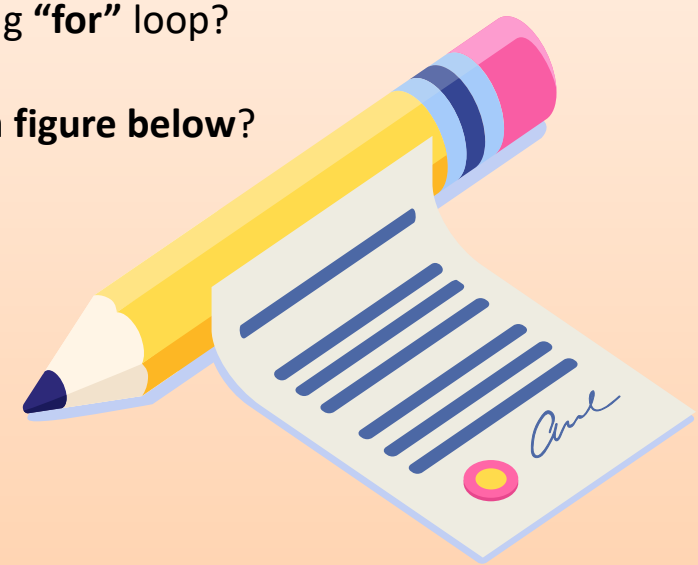
Quiz

Homework

- Calculate **average** of **10** elements in ***int*** array using **"for"** loop?

- Write a Java code to print the **output** as shown in **figure below**?

```
     *
    ****
   ******
  ********
 **********
************
**************
```



- Using **Nested if** and multiple **else-if** conditions:
  - Print on console **"person is student"** if age is less than **22**
  - Print on console **"person is working"** if age is greater than **22**
  - Print on console **"person has done secondary education"** if age is equal to **16**
  - Print on console **"person has done senior secondary education"** if age is equal to **18**
  - Print on console **"person has done engineering"** if age is equal to **22**

- Declare a **String array** with length **5**
  - Input the 5 object values in an **array**
  - Write the code to print on console the complete **array**